Web Server Architecture for HTTPServer

Summary

This document contains a description of the classes and methods that make up the *HTTPServer* package created by Nicholas Ferraz de Oliveira (nferraz) as a final project for CSC 583 Network Programming in the Fall of 2020, taught by professor Kamyar Dezhgosha at UIS. The final page includes a computer-generated UML diagram taken from IntelliJ IDE which gives a graphical overview of the classes and the simple chain of dependency among them.

The basic structure of the *HTTPServer* is straightforward. The *HTTPServer* class handles the creation and interruption of Servlet objects, which extend the Thread class. The Servlet create fixed thread pools using the *ExecutorService* class to handle incoming HTTP requests. Each HTTP request is then passed to a new *ProcessRequest* object which takes the socket object for the respective connection, parses the request and routes it to the appropriate HTTP verb helper method.

The *HTTPServer* class also handles both an interactive, command line, administrative interface, which includes commands for creating, stopping and restarting new servlets. It also features a non-interactive, traditional parameter passing option through the command line.

Features

- Supports **multiple servlets** and **multiple document roots**.
- Uses a **thread pool** for processing incoming connections.
- Features a fully interactive **command line administrative interface**. Commands include:
  - create [webroot] [port_number] [start]
  - restart [servlet_number]
  - stop [servlet_number|_] --- defaults to most recently created servlet.
  - list servlets
  - exit
- Handles **GET** and **HEAD HTTP requests** for HTML as well as other file types such as .jpg, png and .pdf.
- Provides logging in the **common web log format** to the console, and if available, it will create a log file in a "logs" folder in the root directory.
- Provides a clear and well documented set of classes and methods that allow for easy maintenance and future enhancements.

HTTPServer

This is the principle class of our web server. The *main()* method for *HTTPServer* starts an interactive command line admin interface that allows the user to pass commands for creating, stopping and restarting new servlets and specifying their web root folder and port number. Additionally, this class contains helper methods for parsing and validating the user input and ensure that all parameters are valid.

Class Variables

private int servlet_count

private ArrayList<Servlet> servlets

private boolean interactive_mode

Methods

private void handleArgs(String[] args)

This method handles command line arguments for non-interactive mode.

private static void usage(String type)

This method takes a string with the error type and prints out the correct usage for port and webroot.

private void createServlet(File webroot, int port, Boolean start) throws IOException

This is method handles the creation of new servlets. In addition to a root directory and port number it takes a start flag to indicate whether the servlet should start immediately or not.

private void stopServlet(int servlet_number)

private void stopServlet()

This method takes the number of the servlet number and stops that servlet. If the servlet number is not provided it will stop the last servlet to be created.

private void restartServlet(int servlet_number)

Checks if the servlet is not currently running and if so creates another servlet with the same parameters to replace it.

private void handleInput(String input) throws IOException

Parses the input in interactive mode and calls the appropriate method.

private void listServlets()

Prints out the status---i.e., running/not running---, port and root directory for all servlets.

private static void listCommands()

Lists all available command line options with usage and example.

private int getAvailablePort(int port)

Checks if the port is available otherwise it gets the next available port starting from 80.

private File getWebroot(String path)

Returns the root directory for the servlet. In interactive mode it will ask the user to reenter the path if it doesn't exist.

Servlet

This class extends the Thread class, which allows new Servlets to be created and run as threads from the *HTTPServer* class. In addition, it also has its own fixed thread pool using the *ExecutorService* class to help handle concurrent HTTP requests.

Class Variables

private ExecutorService thread_pool = Executors.newFixedThreadPool(20)

File webroot

int port

private Boolean run

private int servlet_number

private FileHandler fileHandler

private ServerSocket mainSocket

private Logger auditLog

private Logger errorLog

Methods

public Servlet(File webroot, int port, int servlet_number)

Class constructor. Sets the initial values for certain variables.

public void interrupt()

Attempts to interrupt the thread as gracefully as possible and sets the run variable to false.

public void run()

Mandatory since it extends the Thread class. This is

private void setAuditLogHandler()

Creates the fileHandler and binds it to the auditLog if the directory logs exits. Otherwise it won't be set and the auditLog will log to the console only.

public boolean isRunning()

Returns the run variable, which is the flag variable to determine if this thread should run.

void getStatus()

Prints the web root directory, port number and status of the thread---i.e., is it running or not.

ProcessRequest

This class extends the Thread class, which allows new Servlets to be created and run as threads from the *HTTPServer* class. In addition, it also has its own fixed thread pool using the *ExecutorService* class to help handle concurrent HTTP requests.

Class Variables

private static File webroot

private final Socket socket

private Logger auditLog

Methods

public ProcessRequest(File webroot, Socket socket, Logger auditLog)

The only constructor for the class.

public void run()

Obligatory implementation as the class implements Runnable. Commonly known as "the thread equivalent to the main method".

public void routeRequest(String[] requestHeader) throws IOException

This is method handles parsing the requests and then "routing" it to the appropriate HTTP verb.

private void HTTPGet(OutputStream outputStream, Writer writer, File file, String mimeType, String version) throws IOException

An implementation of the HTTP GET verb.

private void HTTPHead(Writer writer, File file, String mimeType, String version) throws IOException

An implementation of the HTTP HEAD verb.

public void invalidVerb(String verb)

Handles error for verbs that have not been implemented.

public void resourceNotFound(Writer writer, String version) throws IOException

Returns a 404 page and header if the resource cannot be found.

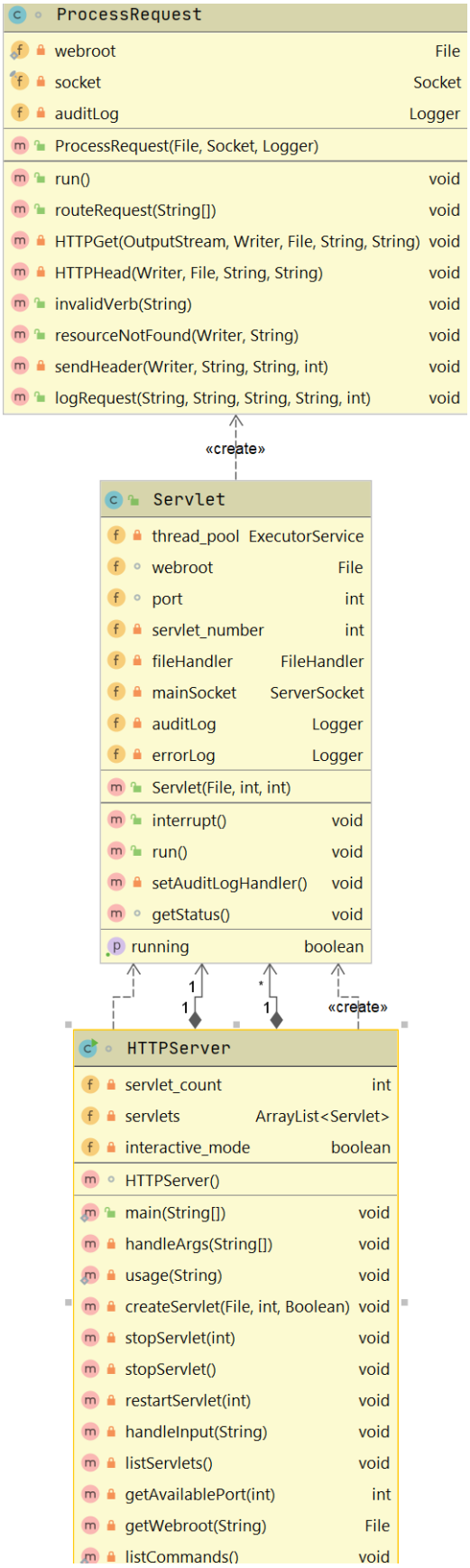
private void sendHeader(Writer writer, String responseCode, String mimeType, int length) throws IOException

Writes the header response to the client.

public void logRequest(String verb, String fileName, String version, String code, int bytes)

Logs the request to log/server_log.log and/or the console.

The last page contains a UML diagram of the HTTPServer package generated by IntelliJ IDE.


ProcessRequest
webroot File
socket Socket
auditLog Logger
ProcessRequest(File, Socket, Logger)
run() void
routeRequest(String[]) void
HTTPGet(OutputStream, Writer, File, String, String) void
HTTPHead(Writer, File, String, String) void
invalidVerb(String) void
resourceNotFound(Writer, String) void
sendHeader(Writer, String, String, int) void
logRequest(String, String, String, String, int) void
«create»
Servlet
thread_pool ExecutorService
webroot File
port int
servlet_number int
fileHandler FileHandler
mainSocket ServerSocket
auditLog Logger
errorLog Logger
Servlet(File, int, int)
interrupt() void
run() void
setAuditLogHandler() void
getStatus() void
running boolean
1
1
*
1
«create»
HTTPServer
servlet_count int
servlets ArrayList<Servlet>
interactive_mode boolean
HTTPServer()
main(String[]) void
handleArgs(String[]) void
usage(String) void
createServlet(File, int, Boolean) void
stopServlet(int) void
stopServlet() void
restartServlet(int) void
handleInput(String) void
listServlets() void
getAvailablePort(int) int
getWebroot(String) File
listCommands() void